

Static and Dynamic Data Structures and BSTs

CS1200: Introduction to Algorithms and their Limitations

Lecture 5

Fall 2026

Harvard SEAS

Put away your devices

Static abstract data types

Recap: the Static Predecessors & Successors problem

Last time: INTERVAL SCHEDULING-DECISION $\leq$ SORTING. Then a new interval $[a^*, b^*]$ arrived. Can we solve the problem *without re-sorting*?

Def. 5.1. A **static abstract data type** is a quadruple $\Pi = (I, O, Q, f)$: inputs $x \in I$, outputs O , a set Q of queries, and for each x and $q \in Q$ a set $f(x, q) \subseteq O$ of *valid answers*.

Input: an array of key-value pairs $x = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, each $K_i \in \mathbb{R}$. **Queries:**

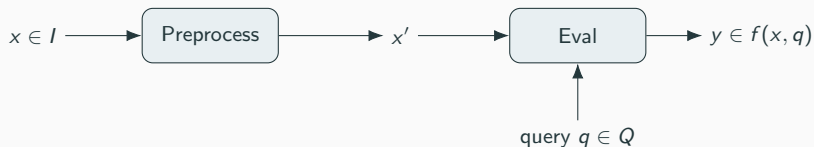
- $\text{search}(K)$: some (K_i, V_i) with $K_i = K$.
- $\text{predecessor}(K)$: some (K_i, V_i) with $K_i = \max\{K_j : K_j < K\}$.
- $\text{successor}(K)$: some (K_i, V_i) with $K_i = \min\{K_j : K_j > K\}$.

Keys $\{2, 3, 5, 7\}$. **What is** $\text{predecessor}(5)$? 3. Both are *strict*: $<$ and $>$, not $\leq$ and $\geq$.

Data Structures

Data structures for an abstract data type

A (static) data structure for $\Pi = (I, O, Q, f)$ is a pair of algorithms (Preprocess, Eval) — Def. 5.2.



- Preprocess runs *once* on the data; Eval runs on *every* query, using the encoding x' , not the original input x .
- **Requirement:** $\text{Eval}(\text{Preprocess}(x), q) \in f(x, q)$ whenever $f(x, q) \neq \emptyset$, and $= \perp$ when $f(x, q) = \emptyset$.

In groups: a data structure for Predecessors & Successors

Is a sorted array a data structure for Predecessors & Successors?

Data structure = (Preprocess, Eval). Sorted array = $x' = \text{Preprocess}(x)$: the format both algorithms use.

Fill in the algorithms and worst-case times.

	algorithm	time
Preprocess(x)	MergeSort the pairs by key $\rightarrow x'$	$O(n \log n)$
Eval(x' , (search, K))	binary search x' for a key = K	$O(\log n)$
Eval(x' , (predecessor, K))	binary search for $i = \max\{j : K'_j < K\}$	$O(\log n)$
Eval(x' , (successor, K))	binary search for $i = \min\{j : K'_j > K\}$	$O(\log n)$

Theorem 5.3. Pay $O(n \log n)$ once; answer any number of queries at $O(\log n)$ each. Space $O(n)$.

Predecessor on a sorted array

Predecessor(x', K)

```
0  lo = 0; hi = n - 1; ans =  $\perp$ 
1  while lo  $\leq$  hi:
2      mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
3      if  $K'_{mid} < K$ :
4          ans = ( $K'_{mid}, V'_{mid}$ )
5          lo = mid + 1
6      else:
7          hi = mid - 1
8  return ans
```

$x' = ((K'_0, V'_0), \dots, (K'_{n-1}, V'_{n-1}))$ sorted by key.

- **Invariant:** every key left of lo is $< K$; every key right of hi is $\geq K$; ans is the largest key $< K$ seen so far.
- Each iteration halves hi - lo: $O(\log n)$.
- The test is $<$, not $\leq$: a key equal to K is treated as *right* of hi and never becomes ans.
- ans stays $\perp$ iff $K'_0 \geq K$.

Predecessor on a sorted array: example

predecessor(45) on keys 29, 29, 41, 45, 50, 63, 71.

0	1	2	3	4	5	6
29	29	41	45	50	63	71

lo	hi	mid	test	effect
0	6	3	45 < 45? no	hi = 2
0	2	1	29 < 45? yes	ans = 29, lo = 2
2	2	2	41 < 45? yes	ans = 41, lo = 3
3	2	—	lo > hi	return 41

Three probes for seven keys. 45 is *in* the array and is skipped: predecessor is strict.

When the data changes

Data arrives over time

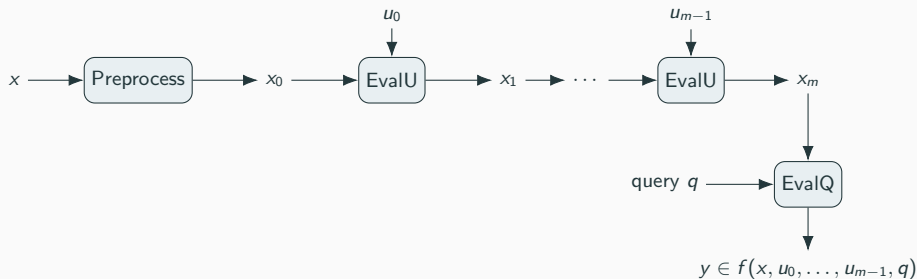
What if new data keeps coming?

A **dynamic abstract data type** (Def. 5.4) adds **updates** to the picture. For DYNAMIC PREDECESSORS & SUCCESSORS, over a multiset S (duplicates allowed) of key-value pairs:

- $\text{insert}(K, V)$: add one copy of (K, V) to S .
- $\text{delete}(K)$: remove one pair with key K , if any.
- queries: search, predecessor, successor — now answered about the *current* S .

One structure, many operations

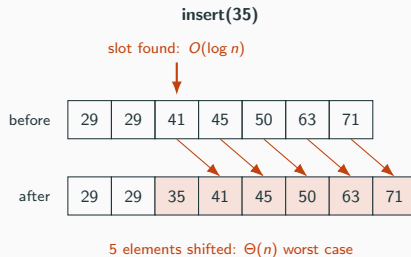
A **(dynamic) data structure** for $\Pi = (I, O, U, Q, f)$ is a *triple of algorithms* (Preprocess, EvalU, EvalQ)
— Def. 5.5. Each update transforms the encoding; each query reads it.



Correctness: the answer must be valid for the multiset S that $u_0, \dots, u_{m-1}$ have built — not for the original x .

The dynamic sorted array data structure

query or update	sorted array
search / predecessor / successor	$O(\log n)$
min / max	$O(1)$
rank	$O(\log n)$
insert	$O(n)$
delete	$O(n)$



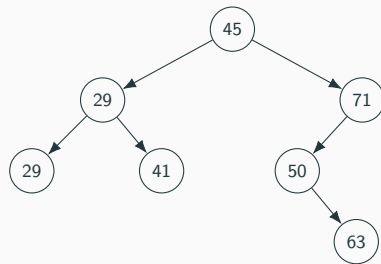
Finding *where* the pair goes: $O(\log n)$. *Putting* it there: every larger element moves one slot.

Binary search trees

Binary search trees

A **BST** (Def. 5.6) is defined recursively. The empty BST is $\emptyset$. A nonempty BST has a root v with

- a key K_v and a value V_v ,
- a left subtree T_L (possibly empty), rooted at the left child v_L ,
- a right subtree T_R (possibly empty), rooted at v_R .



The BST Property

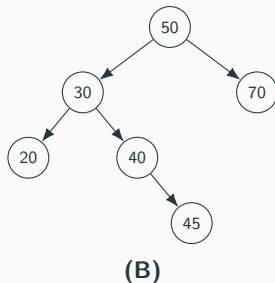
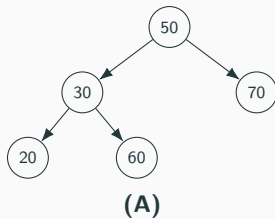
Every key in T_L — the child *and all of its descendants* — is $\leq K_v$; every key in T_R and all of its descendants is $\geq K_v$.

The tree stores the multiset $\{29, 29, 41, 45, 50, 63, 71\}$ — duplicate keys are fine, which is why the property uses $\leq$ and $\geq$.

Def. 5.7. The **height** h of a BST is the length of the longest path from the root to a leaf; the empty tree has height -1 . So $h(T) = 1 + \max\{h(T_L), h(T_R)\}$.

In groups: is it a BST, and what is its height?

For each tree: does the BST Property hold (if not, at which vertex)? What is h ?

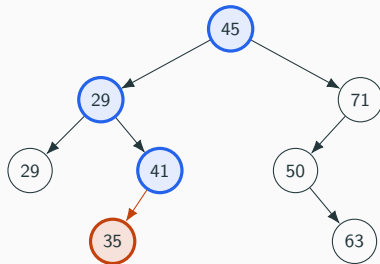


(A) not a BST: fails at the root ($60 > 50$); $h = 2$. (B) a BST; $h = 3$. (C) a BST; $h = 0$.

Insert: walk down, add a leaf

Insert($T, (K, V)$)

```
0  if  $T = \emptyset$ :
1    return a new single vertex  $(K, V)$ 
2  else:
3    let  $v$  be the root of  $T$ 
4    if  $K \leq K_v$ :
5       $T'_L = \text{Insert}(T_L, (K, V))$ 
6       $T' = T$  with left subtree  $T'_L$ 
7      return  $T'$ 
8    else:
9       $T'_R = \text{Insert}(T_R, (K, V))$ 
10      $T' = T$  with right subtree  $T'_R$ 
11     return  $T'$ 
```



Insert 35: $35 \leq 45$ go left, $35 > 29$ go right, $35 \leq 41$ go left, empty, add it as a leaf.

Insert: Proof of correctness

What proof strategy should we use to prove Insert correct? By **strong induction** on the **height h** of T .

Claim (*):

For every BST T of height $\leq h$ storing multiset S , $\text{Insert}(T, (K, V))$ returns a valid BST storing $S \cup \{(K, V)\}$.

- **Base case $h = -1$:** $T = \emptyset$. We return a single vertex with no children: a valid BST storing $\{(K, V)\}$.
- **Inductive hypothesis:** Assume (*) for every tree of height $\leq h$; show it for every tree of height $\leq h + 1$.
- Take T of height $h + 1$ with root v . Say $K \leq K_v$ (the other case is symmetric). T_L has height $\leq h$, so by hypothesis T'_L is a valid BST storing $S_L \cup \{(K, V)\}$.
- **Does v still satisfy the property?** T_R is untouched. Every key in T'_L is either an old key of T_L (so $\leq K_v$, since T was a BST) or K itself (so $\leq K_v$, by the case assumption). □

Let $T_{\text{insert}}(h)$ be the worst-case time on a tree of height $\leq h$.

- Lines 0–3 cost $O(1)$: look at the root, compare one key.
- Exactly one of lines 5, 9 runs, on a subtree of height $\leq h - 1$.

$$T_{\text{insert}}(h) = T_{\text{insert}}(h - 1) + O(1) \quad \implies \quad T_{\text{insert}}(h) = O(h).$$

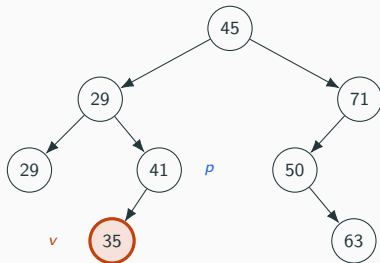
Search, min, max, predecessor and successor are the same length walk with different conditions, all $O(h)$.

Deletion

The two easy cases

Insert always adds a *leaf*. Delete has to cope with a vertex anywhere in the tree. Let v be the vertex with $K_v = K$, and p its parent; split on how many children v has.

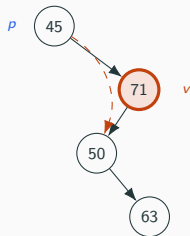
Case 0: v is a leaf. Delete p 's pointer to v .



Delete 35.

Both are $O(1)$ once v is found, and both keep the BST Property: no ancestor–descendant relation among the *remaining* vertices changed.

Case 1: v has one child. Re-point p 's pointer from v to that child.

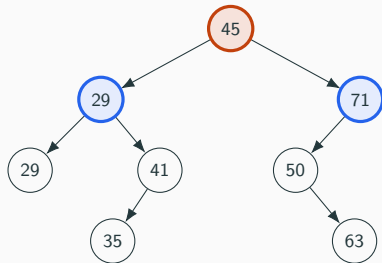


Delete 71: the subtree at 50 moves up.

Two children: the hard case

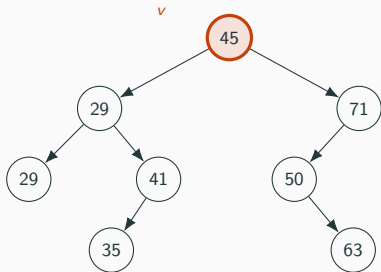
Consider deleting the root, $v = 45$. It has *two* children.

- **Promote 29?** It already has two children (29 and 41), and would also have to hold the subtree at 71: **three subtrees, two slots**. The *structure* breaks.
- **Promote 71?** It has one child, so the structure is fine — but the orphaned subtree at 29 lands to the *right* of 71, and every key in it (29, 29, 35, 41) is < 71 . **The BST Property breaks.**



In groups: how do we delete 45?

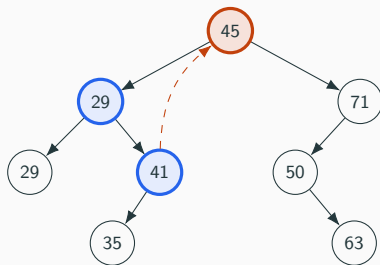
So: neither subtree can simply move up, and promoting a child breaks the shape or the ordering. What *can* we do?



Rather than move a subtree, **move one key-value pair into v** and delete the vertex it came from.

- Which vertices could stand in for 45 without breaking the BST Property?
- How do we know that the vertex we pick has at most one child?

Case 2: v has two children



Take $w =$ the **maximum of v 's left subtree**: go left once, then right as far as you can. Here $w = 41$.

- $K_w \geq$ every key in the left subtree (it is the max), and $K_w \leq K_v \leq$ every key in the right subtree.
So writing (K_w, V_w) into v keeps the BST Property.
- w has *no right child* — that is how we found it — so deleting w is Case 0 or Case 1. Here w has the child 35.
- min of the right subtree (50) works just as well; we follow the book.

Runtime: one search $O(h)$ + one max $O(h)$ + $O(1)$ rewiring = $O(h)$.

(Theorem 5.9)

Theorem 5.8

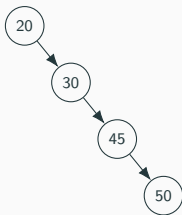
Theorem 5.8. On a BST of height h , all DYNAMIC PREDECESSORS & SUCCESSORS operations — queries *and* updates — plus min and max, run in time $O(h)$.

	sorted array	BST
search / predecessor / successor	$O(\log n)$	$O(h)$
min / max	$O(1)$	$O(h)$
insert	$O(n)$	$O(h)$
delete	$O(n)$	$O(h)$

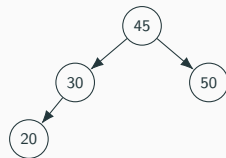
Everything is $O(h)$. So how does h compare to n ?

Keeping the tree short

$O(h)$ is only as good as h



inserted in sorted order: $h = n - 1$



same keys, $h = \Theta(\log n)$

Same data, same algorithms, same $O(h)$ bound — and $O(n)$ versus $O(\log n)$ in practice.

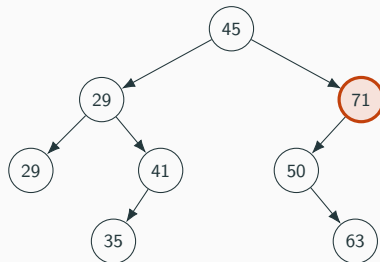
So: can we force $h = O(\log n)$, while keeping the BST property?

AVL trees

Def. 5.10. An **AVL tree** is a BST that is

1. **height-balanced:** every pair of siblings has heights differing by at most 1 (empty tree: height -1), and
2. **height-augmented:** every vertex stores the height of its own subtree.

Lemma 5.11. A height-balanced tree on n vertices has $h \leq 2 \log_2 n$. **Theorem 5.14.** So every operation in our table runs in $O(\log n)$.



Our tree is **not** AVL: at 71, the left subtree has height 1 and the right is empty (height -1).